Tratamiento de imagen: diagnóstico y corrección con OpenCV

Informe de resultados

Experiencia IMAGEN · Semana 8

Estudiante: Ruby Dayana Cárdenas Gómez

Institución: Universidad de Cundinamarca

Programa: CADI Percepción Computacional – Especialización en Inteligencia Artificial

Fecha: Octubre de 2026

1. Introducción

En la Semana 7 aprendí que una imagen es una matriz de números y que OpenCV permite transformarla. En esta semana di el siguiente paso: antes de transformar hay que diagnosticar. La idea de la actividad es que un tratamiento de imagen no se aplica "porque sí", sino que parte de un problema medido, una decisión técnica y una evaluación honesta de si el resultado mejoró o no.

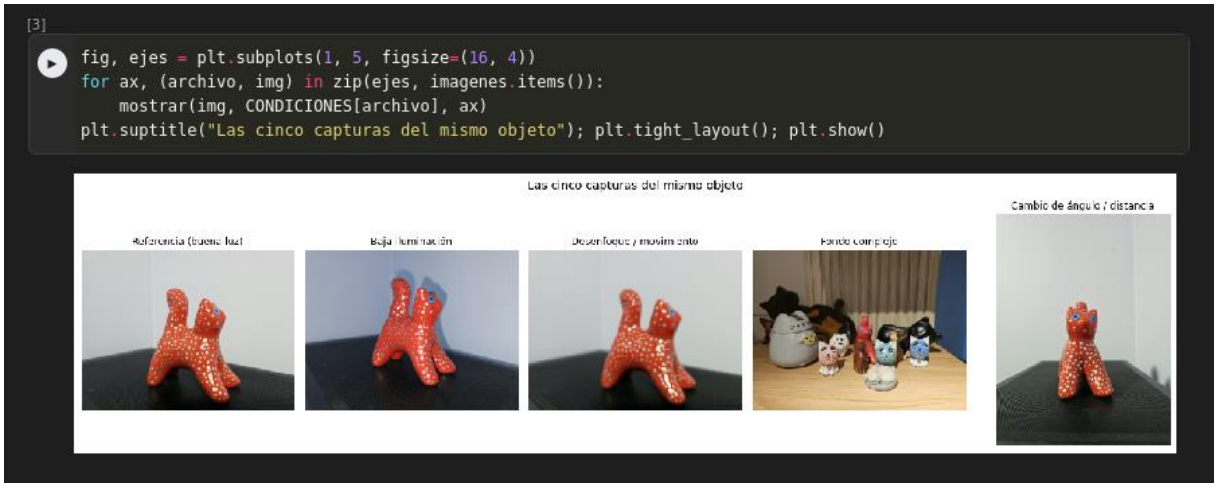
Para eso fotografié un mismo objeto, un gato de cerámica rojo con puntos plateados, en cinco condiciones distintas, medí sus características con Python y OpenCV en Google Colab, identifiqué la imagen más problemática y le apliqué un tratamiento coherente con ese problema. El cuaderno con todo el código y sus salidas está en: **https://github.com/RubyDayana/tratamiento-imagen-semana8**.

2. Las imágenes y sus condiciones de captura

Las cinco fotos se tomaron con el celular, el mismo día, sobre una mesa negra con pared clara detrás (excepto la de fondo complejo). Las imágenes llegaron al computador por WhatsApp, que las recomprime a 1600 × 1200 píxeles; lo menciono porque afecta el tamaño de archivo y agrega artefactos de compresión.

Archivo	Condición	Cómo se tomó
referencia.jpg	Referencia	Luz natural, objeto centrado, fondo liso. Es la foto "ideal" contra la que comparo las demás.
baja_luz.jpg	Baja iluminación	Luz principal apagada; el celular compensó con flash, por eso aparece una sombra dura y un tono azulado en la pared.
desenfoque.jpg	Desenfoque / movimiento	Enfoque fijado en otro punto antes de disparar; el objeto queda fuera de foco.
fondo_complejo.jpg	Fondo complejo	El mismo gato entre una colección de figuras de gatos sobre una mesa de madera.
angulo.jpg	Cambio de ángulo	Foto vertical, de frente y desde más cerca, con el objeto mirando a la cámara.

Tabla 1. Descripción de las cinco capturas.

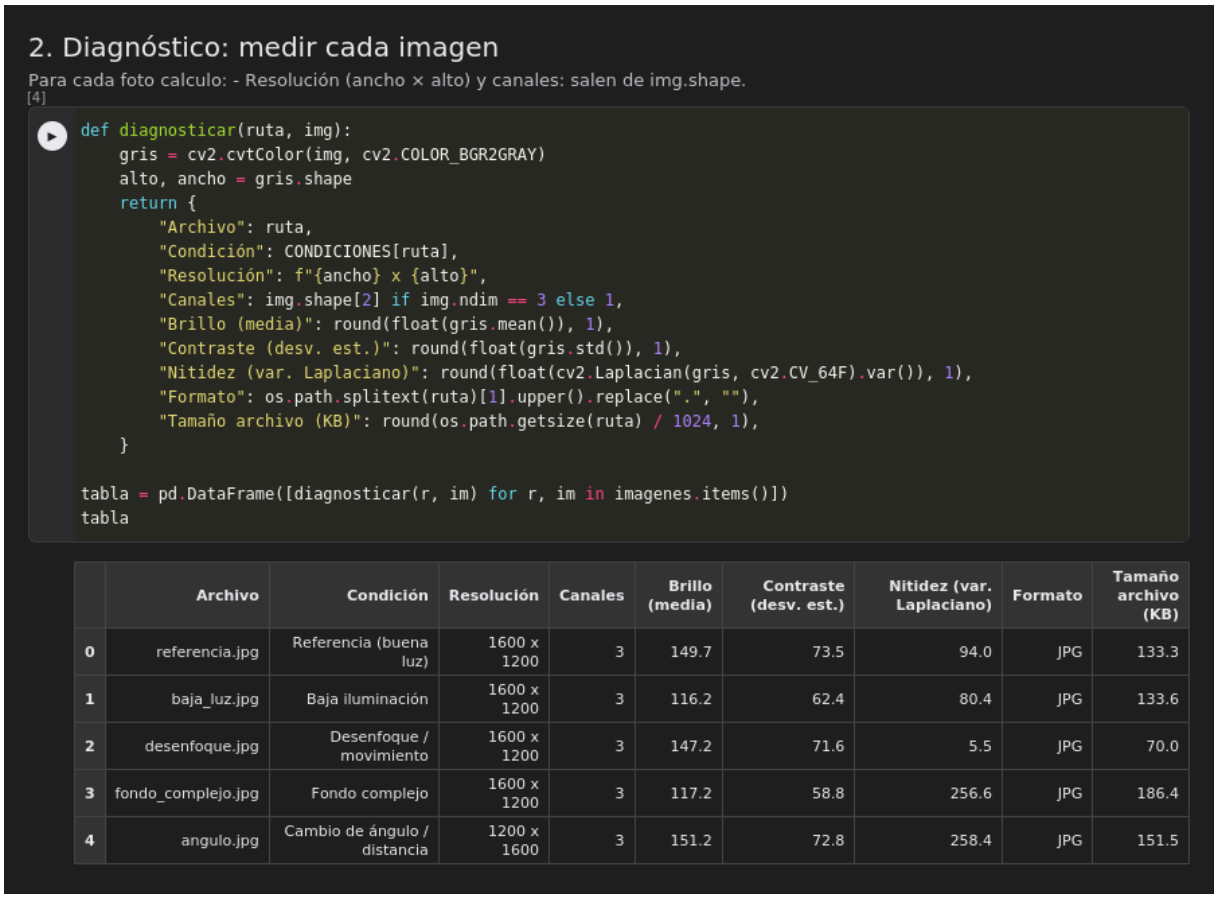


Captura 1. Las cinco fotos del mismo objeto tal como las cargó el cuaderno.

3. Diagnóstico

3.1 Qué se midió y cómo

Para cada foto el cuaderno calcula seis cosas. La **resolución** y los **canales** salen directamente de `img.shape`. El **brillo** es el promedio del canal de luminosidad (la imagen en gris, de 0 negro a 255 blanco). El **contraste** es la desviación estándar de ese gris: si es baja, todos los píxeles se parecen y la foto se ve plana. La **nitidez** se mide con la varianza del Laplaciano: el Laplaciano resalta los bordes, así que una imagen desenfocada, que casi no tiene bordes, da un valor bajo. Por último, el **formato y tamaño del archivo** se leen del disco con `os.path.getsize`.



2. Diagnóstico: medir cada imagen

Para cada foto calculo: - Resolución (ancho × alto) y canales: salen de img.shape.

[4]

```
def diagnosticar(ruta, img):
    gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    alto, ancho = gris.shape
    return {
        "Archivo": ruta,
        "Condición": CONDICIONES[ruta],
        "Resolución": f"{ancho} x {alto}",
        "Canales": img.shape[2] if img.ndim == 3 else 1,
        "Brillo (media)": round(float(gris.mean()), 1),
        "Contraste (desv. est.)": round(float(gris.std()), 1),
        "Nitidez (var. Laplaciano)": round(float(cv2.Laplacian(gris, cv2.CV_64F).var()), 1),
        "Formato": os.path.splitext(ruta)[1].upper().replace(".", ""),
        "Tamaño archivo (KB)": round(os.path.getsize(ruta) / 1024, 1),
    }

tabla = pd.DataFrame([diagnosticar(r, im) for r, im in imagenes.items()])
tabla
```

	Archivo	Condición	Resolución	Canales	Brillo (media)	Contraste (desv. est.)	Nitidez (var. Laplaciano)	Formato	Tamaño archivo (KB)
0	referencia.jpg	Referencia (buena luz)	1600 x 1200	3	149.7	73.5	94.0	JPG	133.3
1	baja_luz.jpg	Baja iluminación	1600 x 1200	3	116.2	62.4	80.4	JPG	133.6
2	desenfoque.jpg	Desenfoque / movimiento	1600 x 1200	3	147.2	71.6	5.5	JPG	70.0
3	fondo_complejo.jpg	Fondo complejo	1600 x 1200	3	117.2	58.8	256.6	JPG	186.4
4	angulo.jpg	Cambio de ángulo / distancia	1200 x 1600	3	151.2	72.8	258.4	JPG	151.5

Captura 2. Función de diagnóstico y tabla comparativa generada por el cuaderno.

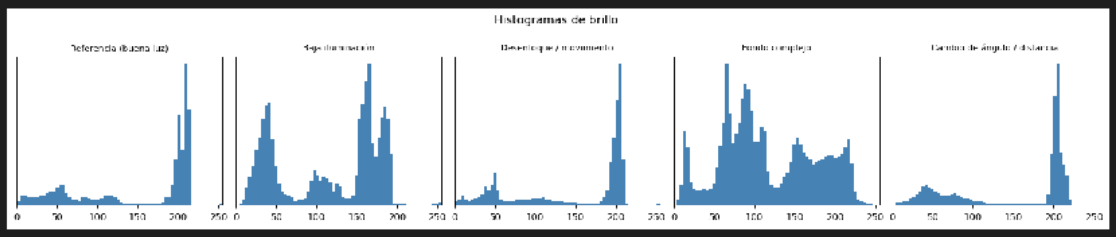
3.2 Tabla comparativa

Imagen	Resolución	Brillo	Contraste	Nitidez	Tamaño (KB)
Referencia	1600 × 1200	149,7	73,5	94,0	133,3
Baja iluminación	1600 × 1200	116,2	62,4	80,4	133,6
Desenfoque	1600 × 1200	147,2	71,6	5,5	70,0
Fondo complejo	1600 × 1200	117,2	58,8	256,6	186,4
Cambio de ángulo	1200 × 1600	151,2	72,8	258,4	151,5

Tabla 2. Resultados del diagnóstico (todas con 3 canales, formato JPG).

[5]

```
# Histogramas de brillo: muestran cómo se reparten los píxeles entre oscuro (0) y claro (255)
fig, ejes = plt.subplots(1, 5, figsize=(16, 3.2))
for ax, (archivo, img) in zip(ejes, imagenes.items()):
    gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ax.hist(gris.ravel(), bins=64, range=(0, 255), color="steelblue")
    ax.set_title(CONDICIONES[archivo], fontsize=9); ax.set_xlim(0, 255); ax.set_yticks([])
plt.suptitle("Histogramas de brillo"); plt.tight_layout(); plt.show()
```



Captura 3. Histogramas de brillo de las cinco imágenes.

3.3 Problemas encontrados

Desenfoque: es el problema más grave y el más claro en los números. La nitidez cae de 94,0 a 5,5, apenas el 5,9 % de la referencia, mientras que brillo y contraste se mantienen casi iguales (98 % y 97 %). Es decir, la foto tiene la luz correcta pero no tiene bordes. Además el archivo pesa casi la mitad (70 KB frente a 133 KB), porque una imagen sin detalle se comprime mucho más: el tamaño del archivo resultó ser un indicador inesperado de nitidez.

Baja iluminación: el brillo bajó a 116 (78 % de la referencia) y el contraste a 62, pero menos de lo que esperaba. La explicación está en la propia foto: el celular detectó poca luz y disparó el flash, así que la imagen no quedó oscura sino con una sombra dura y una dominante azul en la pared. La cámara ya "trató" la imagen antes que yo; eso cambia el problema real de "falta de luz" a "iluminación no uniforme y color alterado".

Fondo complejo: aquí las métricas engañan. La nitidez es la más alta de todas (256,6), pero no por el gato sino porque la madera, las figuras y la cortina llenan la imagen de bordes. El brillo bajo (117) también viene del fondo, no del objeto. Las métricas globales no sirven para evaluar un objeto cuando el fondo domina la escena; haría falta recortar la región de interés.

Cambio de ángulo: no es un problema de calidad (brillo, contraste y nitidez son iguales o mejores que la referencia), sino de contenido: la orientación es vertical (1200 × 1600) y el objeto se ve desde otro punto de vista, lo que importaría para un modelo entrenado con fotos de frente.

```
[6]
# Comparación numérica frente a la referencia
ref = tabla.iloc[0]   # la primera fila es la referencia
print("Comparado con la imagen de referencia:\n")
for _, fila in tabla.iloc[1:].iterrows():
    print(f"{fila['Condición']:30s} brillo {fila['Brillo (media)']:6.1f} ({fila['Brillo (media)']/ref['Brillo
(media)']*100:5.1f} % de la ref.) | "
          f"contraste {fila['Contraste (desv. est.)']:5.1f} ({fila['Contraste (desv. est.)']/ref['Contraste
(desv. est.)']*100:5.1f} %) | "
          f"nitidez {fila['Nitidez (var. Laplaciano)']:7.1f} ({fila['Nitidez (var. Laplaciano)']/ref['Nitidez
(var. Laplaciano)']*100:5.1f} %)")

Comparado con la imagen de referencia:

Baja iluminación               brillo  116.2 ( 77.6 % de la ref.) | contraste  62.4 ( 84.9 %) | nitidez    80.4 (
85.5 %)
Desenfoque / movimiento        brillo  147.2 ( 98.3 % de la ref.) | contraste  71.6 ( 97.4 %) | nitidez     5.5 (
5.9 %)
Fondo complejo                 brillo  117.2 ( 78.3 % de la ref.) | contraste  58.8 ( 80.0 %) | nitidez   256.6
(273.0 %)
Cambio de ángulo / distancia   brillo  151.2 (101.0 % de la ref.) | contraste  72.8 ( 99.0 %) | nitidez   258.4
(274.9 %)
```

Captura 4. Comparación numérica de cada imagen frente a la referencia.

4. Problema seleccionado y tratamiento

Elegí tratar `desenfoque.jpg`, porque es la única imagen cuyo problema es inequívoco en la tabla (una sola métrica, la nitidez, cae al 6 %) y porque existe un tratamiento directamente relacionado con esa métrica: la **máscara de desenfoque** (*unsharp mask*). La técnica consiste en restar a la imagen una versión borrosa de sí misma para quedarse con los bordes, y sumarlos de nuevo amplificados: `tratada = original + fuerza × (original − borrosa)`. En OpenCV se hace con `GaussianBlur` y `addWeighted`. No elegí la de baja luz porque, como expliqué, el flash ya la había corregido parcialmente, y la de fondo complejo porque su problema no se resuelve con un filtro global.

[6]

```
# Comparación numérica frente a la referencia
ref = tabla.iloc[0]   # la primera fila es la referencia
print("Comparado con la imagen de referencia:\n")
for _, fila in tabla.iloc[1:].iterrows():
    print(f"{fila['Condición']:30s} brillo {fila['Brillo (media)']:6.1f} ({fila['Brillo (media)']/ref['Brillo
(media)']*100:5.1f} % de la ref.) | "
          f"contraste {fila['Contraste (desv. est.)']:5.1f} ({fila['Contraste (desv. est.)']/ref['Contraste
(desv. est.)']*100:5.1f} %) | "
          f"nitidez {fila['Nitidez (var. Laplaciano)']:7.1f} ({fila['Nitidez (var. Laplaciano)']/ref['Nitidez
(var. Laplaciano)']*100:5.1f} %)")
```

```
Comparado con la imagen de referencia:

Baja iluminación               brillo  116.2 ( 77.6 % de la ref.) | contraste  62.4 ( 84.9 %) | nitidez    80.4 (
85.5 %)
Desenfoque / movimiento        brillo  147.2 ( 98.3 % de la ref.) | contraste  71.6 ( 97.4 %) | nitidez     5.5 (
5.9 %)
Fondo complejo                 brillo  117.2 ( 78.3 % de la ref.) | contraste  58.8 ( 80.0 %) | nitidez   256.6
(273.0 %)
Cambio de ángulo / distancia   brillo  151.2 (101.0 % de la ref.) | contraste  72.8 ( 99.0 %) | nitidez   258.4
(274.9 %)
```

[7]

```
# >>> CAMBIA ESTAS DOS LÍNEAS SEGÚN TU DIAGNÓSTICO <<<
ARCHIVO_PROBLEMA = "desenfoque"       # imagen que voy a tratar (sin extensión)
TRATAMIENTO = "nitidez"               # "iluminacion" (baja luz) o "nitidez" (desenfoque)

ARCHIVO_PROBLEMA = [r for r in imagenes if r.startswith(ARCHIVO_PROBLEMA)][0]
original = imagenes[ARCHIVO_PROBLEMA]

def corregir_iluminacion(img, gamma=0.7, clip=3.0):
    """CLAHE sobre el canal L (luminosidad) del espacio LAB + corrección gamma.
    Así se aclara y se recupera contraste sin alterar los colores."""
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    L, A, B = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip, tileGridSize=(8, 8))
    L = clahe.apply(L)
    img_clahe = cv2.cvtColor(cv2.merge([L, A, B]), cv2.COLOR_LAB2BGR)
    # gamma < 1 aclara las zonas oscuras más que las claras
    tabla_gamma = np.array([((i / 255.0) ** gamma) * 255 for i in range(256)]).astype("uint8")
    return cv2.LUT(img_clahe, tabla_gamma)

def corregir_nitidez(img, fuerza=1.5, radio=5):
    """Unsharp mask: original + fuerza * (original - versión borrosa)."""
    borrosa = cv2.GaussianBlur(img, (0, 0), radio)
    return cv2.addWeighted(img, 1 + fuerza, borrosa, -fuerza, 0)

if TRATAMIENTO == "iluminacion":
    tratada = corregir_iluminacion(original)
    nombre_tratamiento = "CLAHE + corrección gamma"
else:
    tratada = corregir_nitidez(original)
    nombre_tratamiento = "Máscara de desenfoque (unsharp mask)"

cv2.imwrite("tratada.jpg", tratada, [cv2.IMWRITE_JPEG_QUALITY, 95])
print("Imagen tratada:", ARCHIVO_PROBLEMA, "->", nombre_tratamiento)
```

```
Imagen tratada: desenfoque.jpg -> Máscara de desenfoque (unsharp mask)
```

Captura 5. Funciones de tratamiento (CLAHE + gamma para iluminación, unsharp mask para nitidez) y selección de la imagen.

Antes de decidir los parámetros probé nueve combinaciones de fuerza y radio y medí la nitidez de cada resultado. A mayor fuerza, mayor nitidez numérica (hasta 64 con fuerza 2,5), pero visualmente aparecían halos muy marcados alrededor de los puntos plateados. Me quedé con fuerza 1,5 y radio 5, que mejora sin exagerar los artefactos.

[8]

```python
# ¿Qué tan fuerte aplicar el realce? Pruebo varias combinaciones y mido la nitidez de cada una
def nitidez(img):
    return cv2.Laplacian(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), cv2.CV_64F).var()

print(f"Nitidez de la referencia: {nitidez(imagenes[REFERENCIA]):.1f}")
print(f"Nitidez del original desenfocado: {nitidez(original):.1f}")
print()
print(f"{'fuerza':>7s} {'radio':>6s} {'nitidez':>9s}")
for fuerza in [1.0, 1.5, 2.5]:
    for radio in [3, 5, 10]:
        print(f"{fuerza:7.1f} {radio:6d} {nitidez(corregir_nitidez(original, fuerza, radio)):9.1f}")
```

```
Nitidez de la referencia: 94.0
Nitidez del original desenfocado: 5.5

 fuerza  radio   nitidez
    1.0      3      21.9
    1.0      5      22.2
    1.0     10      22.4
    1.5      3      34.0
    1.5      5      34.5
    1.5     10      34.7
    2.5      3      63.7
    2.5      5      64.1
    2.5     10      63.4
```

Captura 6. Exploración de parámetros: nitidez obtenida con cada combinación de fuerza y radio.

5. Comparación antes / después

[7]

```
# >>> CAMBIA ESTAS DOS LÍNEAS SEGÚN TU DIAGNÓSTICO <<<
ARCHIVO_PROBLEMA = "desenfoque"       # imagen que voy a tratar (sin extensión)
TRATAMIENTO = "nitidez"               # "iluminacion" (baja luz) o "nitidez" (desenfoque)

ARCHIVO_PROBLEMA = [r for r in imagenes if r.startswith(ARCHIVO_PROBLEMA)][0]
original = imagenes[ARCHIVO_PROBLEMA]

def corregir_iluminacion(img, gamma=0.7, clip=3.0):
    """CLAHE sobre el canal L (luminosidad) del espacio LAB + corrección gamma.
    Así se aclara y se recupera contraste sin alterar los colores."""
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    L, A, B = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip, tileGridSize=(8, 8))
    L = clahe.apply(L)
    img_clahe = cv2.cvtColor(cv2.merge([L, A, B]), cv2.COLOR_LAB2BGR)
    # gamma < 1 aclara las zonas oscuras más que las claras
    tabla_gamma = np.array([((i / 255.0) ** gamma) * 255 for i in range(256)]).astype("uint8")
    return cv2.LUT(img_clahe, tabla_gamma)

def corregir_nitidez(img, fuerza=1.5, radio=5):
    """Unsharp mask: original + fuerza * (original - versión borrosa)."""
    borrosa = cv2.GaussianBlur(img, (0, 0), radio)
    return cv2.addWeighted(img, 1 + fuerza, borrosa, -fuerza, 0)

if TRATAMIENTO == "iluminacion":
    tratada = corregir_iluminacion(original)
    nombre_tratamiento = "CLAHE + corrección gamma"
else:
    tratada = corregir_nitidez(original)
    nombre_tratamiento = "Máscara de desenfoque (unsharp mask)"

cv2.imwrite("tratada.jpg", tratada, [cv2.IMWRITE_JPEG_QUALITY, 95])
print("Imagen tratada:", ARCHIVO_PROBLEMA, "->", nombre_tratamiento)
```

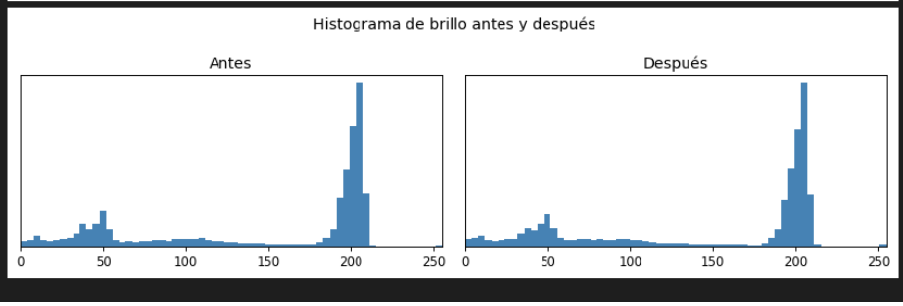
```
Imagen tratada: desenfoque.jpg -> Máscara de desenfoque (unsharp mask)
```

[9]

```
fig, ejes = plt.subplots(1, 3, figsize=(15, 5))
mostrar(imagenes[REFERENCIA], "Referencia", ejes[0])
mostrar(original, f"Antes: {CONDICIONES[ARCHIVO_PROBLEMA]}", ejes[1])
mostrar(tratada, f"Después: {nombre_tratamiento}", ejes[2])
plt.tight_layout(); plt.show()

# Histogramas antes y después
fig, ejes = plt.subplots(1, 2, figsize=(10, 3))
for ax, (im, t) in zip(ejes, [(original, "Antes"), (tratada, "Después")]):
    g = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
    ax.hist(g.ravel(), bins=64, range=(0, 255), color="steelblue"); ax.set_title(t); ax.set_xlim(0, 255);
ax.set_yticks([])
plt.suptitle("Histograma de brillo antes y después"); plt.tight_layout(); plt.show()
```

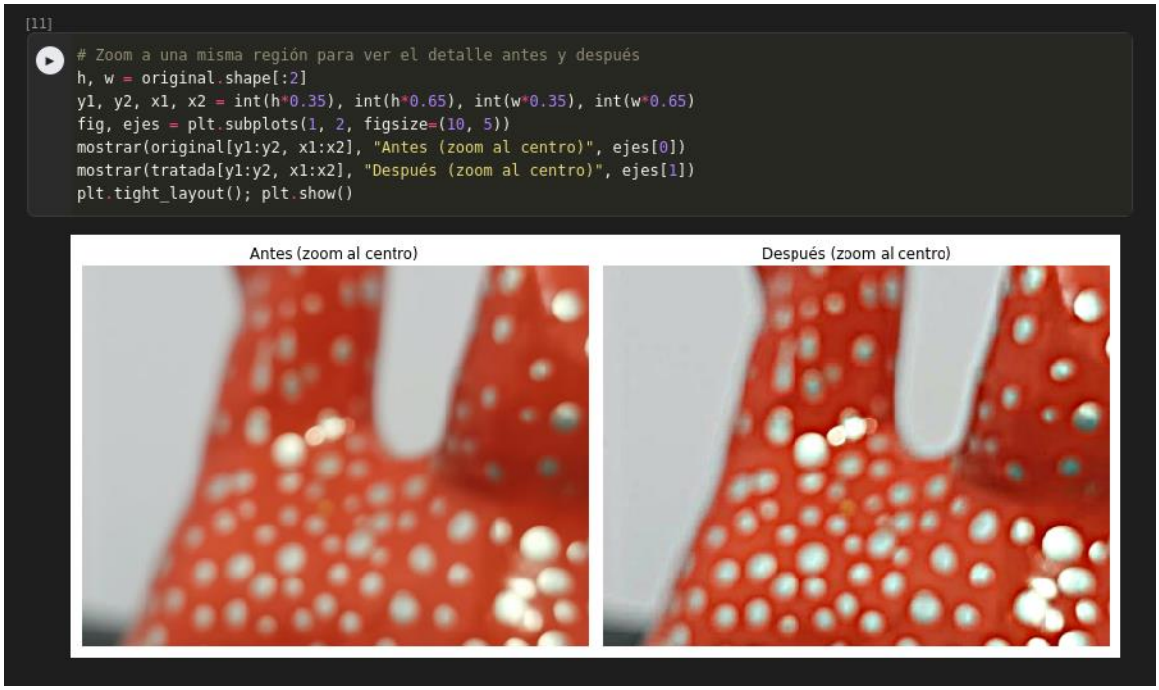




Captura 7. Referencia, imagen desenfocada e imagen tratada, con sus histogramas de brillo.

Métrica	Referencia	Antes	Después	Cambio
Brillo (media)	149,7	147,2	147,2	0,0 %
Contraste (desv. est.)	73,5	71,6	72,5	+1,3 %
Nitidez (var. Laplaciano)	94,0	5,5	34,5	+527 %

Tabla 3. Métricas antes y después del tratamiento (fuerza 1,5, radio 5).



Captura 8. Zoom a la región central antes y después: los puntos plateados y los bordes se definen, pero aparecen halos claros.

6. Análisis de resultados

¿Mejoró de verdad? Sí, pero menos de lo que sugiere el porcentaje. La nitidez se multiplicó por seis (de 5,5 a 34,5) y a simple vista los puntos plateados, el contorno de las patas y la cola se ven más definidos; el histograma apenas cambió, lo que confirma que el tratamiento tocó los bordes y no el brillo general, que era lo que quería. Sin embargo, 34,5 sigue siendo poco más de un tercio de la nitidez de la referencia (94,0), y en el zoom el ojo del gato continúa sin definición.

Por qué no se puede llegar más lejos. La máscara de desenfoque amplifica los bordes que todavía existen en la imagen; no inventa los que la cámara no capturó. Cuando subí la fuerza a 2,5 la nitidez numérica llegó a 64, pero lo que creció fueron los halos blancos alrededor de cada punto, un artefacto, no detalle real. Esto me enseñó que una métrica puede mejorar mientras la imagen empeora visualmente, y que por eso la evaluación debe combinar números y ojos.

Limitaciones encontradas. (1) El realce también amplifica los artefactos de compresión JPEG que dejó WhatsApp. (2) Para un desenfoque fuerte la solución correcta sería una deconvolución que conozca cómo se movió la cámara, algo que no tenía. (3) Las métricas globales (brillo y nitidez de toda la imagen) no describen bien al objeto cuando el fondo pesa más que él, como pasó en la foto de fondo complejo. (4) La foto de “baja luz” mostró que el celular ya aplica tratamientos automáticos (flash, balance de blancos) que cambian el problema antes de que uno lo analice.

7. Conclusiones y recomendaciones

Conclusiones

- El diagnóstico numérico funciona: la tabla identificó sin ambigüedad el desenfoque (nitidez al 6 % de la referencia) y, de paso, reveló que el tamaño del archivo JPG es un indicador indirecto de la cantidad de detalle.
- El tratamiento debe responder a la métrica que falla. Como fallaba la nitidez, apliqué un realce de bordes y comprobé que brillo y contraste quedaron intactos; eso es lo que distingue un tratamiento justificado de aplicar filtros al azar.
- Mejorar una métrica no es lo mismo que mejorar la imagen. Entre fuerza 1,5 y 2,5 el número subió pero la calidad visual bajó por los halos. La evaluación necesita las dos evidencias.
- La captura manda. Ningún filtro recuperó la nitidez de la referencia; el tratamiento recupera parte, pero la mejor corrección de una foto desenfocada es volver a tomarla.

Recomendaciones

- Medir las métricas sobre la región del objeto (recorte o máscara) y no sobre toda la imagen, para que el fondo no distorsione el diagnóstico.
- Transferir las fotos sin recompresión (cable, Drive o "documento" en WhatsApp) para no sumar artefactos JPEG antes del análisis.
- Para la foto de baja luz, desactivar el flash y repetir la captura; luego comparar CLAHE con la corrección gamma, que el cuaderno ya tiene implementadas.
- Como siguiente paso, usar estas cinco condiciones como "aumentación de datos" para entrenar un clasificador y medir cuánto afecta cada condición a la predicción.

8. Enlace al cuaderno

Cuaderno de Google Colab con el código completo, las cinco imágenes y todas las salidas: **https://github.com/RubyDayana/tratamiento-imagen-semana8**.

Archivo: `Semana8_Tratamiento_Imagen_RCardenas.ipynb`.

Referencias

Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

compression.ai. (s. f.). *How to measure the quality of an image*. https://compression.ai/how-to-measure-the-quality-of-an-image/

Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4.ª ed.). Pearson.

OpenCV. (s. f.). *OpenCV tutorials (versión 4.13.0)*. https://docs.opencv.org/4.13.0/d9/df8/tutorial_root.html

Pech-Pacheco, J. L., Cristóbal, G., Chamorro-Martínez, J., & Fernández-Valdivia, J. (2000). Diatom autofocusing in brightfield microscopy: A comparative study. En *Proceedings of the 15th International Conference on Pattern Recognition* (Vol. 3, pp. 314–317). IEEE.